

Міністерство освіти і науки України Національний технічний
університет України "Київський політехнічний інститут імені Ігоря
Сікорського"
Фізико-технічний інститут

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №4

Вивчення криптосистеми RSA та алгоритму електронного підпису;
ознайомлення з методами генерації параметрів для асиметричних
криптосистем

Виконали:

ФБ-21 Редько-Шпак Р.А.

ФБ-21 Серяков В.Л.

ЗМІСТ

Хід роботи	5
RSA.py Покроковий розбір та пояснення	5
Загальна структура rsa.py	5
Блок №1 Імпорти та константи	6
Блок №2 Клас логів	6
Блок №3 Типи ключів для RSA	7
Блок №4 Математичні функції	7
Блок №5 Генерація та перевірка простих чисел	9
Блок №6 Клас RSAMessage	11
Блок №7 Генерація ключа	14
Блок №8 Шифрування/Розшифрування	15
Блок №9 Підпис	17
Узагальнення	18
TEST.py Покроковий розбір та пояснення	18
Загальна структура test.py	18
Блок №0 Допоміжні/візуальні функції	19
Блок/тест №1 Генерація ключів	19
Блок/тест №2 Шифрування/Дешифрування	21
Блок/тест №3 Цифровий підпис	27
Блок/тест №4 Протокол обміну ключами	30
Висновки	33

Мета роботи

Ознайомлення з тестами перевірки чисел на простоту і методами генерації ключів для асиметричної криптосистеми типу RSA; практичне ознайомлення з системою захисту інформації на основі криптосхеми RSA, організація з використанням цієї системи засекреченого зв'язку й електронного підпису, вивчення протоколу розсилання ключів.

Порядок виконання роботи

1. Написати функцію пошуку випадкового простого числа з заданого інтервалу або заданої довжини, використовуючи датчик випадкових чисел та тести перевірки на простоту. В якості датчика випадкових чисел використовуйте вбудований генератор псевдовипадкових чисел вашої мови програмування. В якості тесту перевірки на простоту рекомендовано використовувати тест Міллера-Рабіна із попередніми пробними діленнями. Тести необхідно реалізовувати власноруч, використання готових реалізацій тестів не дозволяється.
2. За допомогою цієї функції згенерувати дві пари простих чисел p, q і p_1, q_1 довжини щонайменше 256 біт. При цьому пари чисел беруться так, щоб $pq \leq p_1q_1$; p і q – прості числа для побудови ключів абонента А, p_1 і q_1 – абонента В.
3. Написати функцію генерації ключових пар для RSA. Після генерування функція повинна повертати та/або зберігати секретний ключ (d, p, q) та відкритий ключ (n, e) . За допомогою цієї функції побудувати схеми RSA для абонентів А і В – тобто, створити та зберегти для подальшого використання відкриті ключі (e, n) , (e_1, n_1) та секретні d і d_1 .
4. Написати програму шифрування, розшифрування і створення повідомлення з цифровим підписом для абонентів А і В. Кожна з

операцій (шифрування, розшифрування, створення цифрового підпису, перевірка цифрового підпису) повинна бути реалізована окремою процедурою, на вхід до якої повинні подаватись лише ті ключові дані, які необхідні для її виконання. За допомогою датчика випадкових чисел вибрати відкрите повідомлення M і знайти криптограму для абонентів A і B , перевірити правильність розшифрування. Скласти для A і B повідомлення з цифровим підписом і перевірити його.

5. За допомогою раніше написаних на попередніх етапах програм організувати роботу протоколу конфіденційного розсилання ключів з підтвердженням справжності по відкритому каналу за допомогою алгоритму RSA. Протоколи роботи кожного учасника (відправника та приймаючого) повинні бути реалізовані у вигляді окремих процедур, на вхід до яких повинні подаватись лише ті ключові дані, які необхідні для виконання. Перевірити роботу програм для випадково обраного ключа $0 < k < n$.

Кожна з наведених операцій повинна бути реалізована у вигляді окремої процедури, інтерфейс якої повинен приймати лише ті дані, які необхідні для її роботи; наприклад, функція `Encrypt()`, яка шифрує повідомлення для абонента, повинна приймати на вхід повідомлення та відкритий ключ адресата (і тільки його), повертаючи в якості результату шифротекст. Відповідно, програмний код повинен містити сім високорівневих процедур: `GenerateKeyPair()`, `Encrypt()`, `Decrypt()`, `Sign()`, `Verify()`, `SendKey()`, `ReceiveKey()`.

Хід роботи

Було реалізовано два Python коди:

1. **rsa.py** - містить всі класи та функції, для реалізації **RSA**;
2. **test.py** - власноруч написані тести, для перевірки всіх можливих сценаріїв роботи коду.

RSA.py | Покроковий розбір та пояснення

Загальна структура rsa.py

```
cryptography > lab.4 > _lab_complete > rsa.py > ...
1 > import random...
4
5 # константи та формат логів
6 DEFAULT_E = 65537 # 2^16 + 1
7 MIN_BIT_LENGTH = 256
8 SMALL_PRIMES = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
9 LOG_FORMATS = { ...
19
20 # клас логів
21 > class CustomFormatter(logging.Formatter): ...
45
46 > def setup_logging(): ...
53
54     setup_logging()
55
56 # типи ключів для RSA
57 > class RSAPublicKey(NamedTuple): ...
60
61 > class RSAPrivateKey(NamedTuple): ...
66
67 # математичні ф-ії
68 > def gcd(a: int, b: int) -> int: ...
75
76 > def extended_gcd(a: int, b: int) -> Tuple[int, int, int]: ...
96
97 > def mod_inverse(a: int, m: int) -> int: ...
112
113 > def mod_pow(base: int, exponent: int, modulus: int) -> int: ...
132
133 > def bit_length(n: int) -> int: ...
137
138 # генерація та перевірка простих чисел
139 > def decompose_number(n: int) -> Tuple[int, int]: ...
148
149 > def trial_division(n: int) -> bool: ...
159
160 > def miller_rabin_test(n: int, k: int = 40) -> bool: ...
184
185 > def generate_prime_candidate(bit_length: int) -> int: ...
191
192 > def generate_prime(bit_length: int, k: int = 40) -> int: ...
198
199 > def generate_safe_prime(bit_length: int, k: int = 40) -> Tuple[int, int]: ...
213
214 # розбиття повідомлень на блоки, підготовки їх до RSA та відновлення оригінального повідомлення з блоків
215 > class RSAMessage: ...
320
321 # генерація ключа
322 > def validate_prime_pair(p: int, q: int, bit_length: int) -> bool: ...
349
350 > def generate_key_pair(bit_length: int = MIN_BIT_LENGTH) -> Tuple[RSAPublicKey, RSAPrivateKey]: ...
399
400 # шифрування/розшифрування
401 > def encrypt_block(block: int, public_key: RSAPublicKey) -> int: ...
409
410 > def decrypt_block(block: int, private_key: RSAPrivateKey) -> int: ...
416
417 > def encrypt(message: Union[str, int, RSAMessage], public_key: RSAPublicKey) -> Tuple[List[int], bool, int]: ...
435
436 > def decrypt(encrypted_blocks: List[int], private_key: RSAPrivateKey, is_text: bool = False, original_bytes_length: int = 0) -> Union[str, int]: ...
465
466 # підпис
467 > def sign(message: Union[str, int, RSAMessage], private_key: RSAPrivateKey) -> List[int]: ...
481
482 > def verify(message: Union[str, int, RSAMessage], signatures: List[int], public_key: RSAPublicKey) -> bool: ...
```

Блок №1 | Імпорти та константи

```
cryptography > lab_4 > _lab_complete > rsa.py > CustomFormatter > format
1 import random
2 import logging
3 from typing import List, Union, Tuple, NamedTuple
4
5 # константи та формат логів
6 DEFAULT_E = 65537 # 2^16 + 1
7 MIN_BIT_LENGTH = 256
8 SMALL_PRIMES = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
9 LOG_FORMATS = {
10     'STEP': '>', # кроки алгоритму і тд.
11     'GENERATE': '[x]', # генерація чогось
12     'ENCRYPT': '[+]', # шифрування
13     'DECRYPT': '[-]', # розшифрування
14     'VERIFY': '[v]', # перевірка (підпису, валідності і тд.)
15     'ERROR': '[!]', # помилки
16     'INFO': '[i]', # загальна інформація
17     'SUCCESS': '[=]' # успіх
18 }
```

Основні імпорти. Задані константи, а саме:

- **e** (відкрита експонента) - за замовчуванням для генерації rsa ключів використовується $e = 2^{16} + 1 = 65537$),
- **MIN_BIT_LENGTH** - мінімальна довжина ключа в бітах,
- **SMALL_PRIMES** - список малих простих чисел для попередньої перевірки.
- Також заданий словник з **LOG_FORMATS** для гарного форматування виводу.

Блок №2 | Клас логів

```
20 # клас логів
21 class CustomFormatter(logging.Formatter):
22     """ клас для форматування логів """
23
24     def format(self, record):
25         msg_lower = record.msg.lower()
26
27         if any(word in msg_lower for word in ["генерація", "generating", "згенеровано"]):
28             prefix = LOG_FORMATS['GENERATE']
29         elif any(word in msg_lower for word in ["крок", "step", "обчислення", "підготовка"]):
30             prefix = LOG_FORMATS['STEP']
31         elif any(word in msg_lower for word in ["розшифрування", "decrypt"]):
32             prefix = LOG_FORMATS['DECRYPT']
33         elif any(word in msg_lower for word in ["шифрування", "encrypt"]):
34             prefix = LOG_FORMATS['ENCRYPT']
35         elif any(word in msg_lower for word in ["перевірка", "валідація", "verify", "validation"]):
36             prefix = LOG_FORMATS['VERIFY']
37         elif record.levelno == logging.ERROR or "помилка" in msg_lower or "error" in msg_lower:
38             prefix = LOG_FORMATS['ERROR']
39         elif any(word in msg_lower for word in ["успішно", "success", "готово", "виконано"]):
40             prefix = LOG_FORMATS['SUCCESS']
41         else:
42             prefix = LOG_FORMATS['INFO']
43
44         return f"{prefix} {record.msg}"
45
46 def setup_logging():
47     """ сетап для всього проекту """
48     handler = logging.StreamHandler()
49     handler.setFormatter(CustomFormatter())
50     logger = logging.getLogger()
51     logger.handlers = [handler]
52     logger.setLevel(logging.INFO)
53
54 setup_logging()
```

Це клас **CustomFormatter**, щоб під час роботи програми ми бачили зрозумілі повідомлення про те, що відбувається.

Блок №3 | Типи ключів для RSA

```
56 # типи ключів для RSA
57 class RSAPublicKey(NamedTuple):
58     e: int # відкрита експонента
59     n: int # модуль
60
61 class RSAPrivateKey(NamedTuple):
62     d: int # секретна експонента
63     p: int # перше просте число
64     q: int # друге просте число
65     n: int # модуль (для зручності)
```

Тут два класи для представлення RSA ключів:

RSAPublicKey (відкритий ключ):

- **e** - відкрита експонента (зазвичай **65537**),
- **n** - модуль (добуток простих чисел **p** і **q**);

RSAPrivateKey (закритий ключ):

d - секретна експонента,

p, q - прості числа, з яких складається модуль,

n - сам модуль (для зручності).

Ми юзали `NamedTuple` для зручного доступ до полів через крапку (`key.e`, `key.n` і тд), це робить код читабельнішим.

Блок №4 | Математичні функції

```
67 # математичні ф-ії
68 > def gcd(a: int, b: int) -> int: ...
69
70 > def extended_gcd(a: int, b: int) -> Tuple[int, int, int]: ...
71
72 > def mod_inverse(a: int, m: int) -> int: ...
73
74 > def mod_pow(base: int, exponent: int, modulus: int) -> int: ...
75
76 > def bit_length(n: int) -> int: ...
```

Цей блок складається з математичних функцій, які необхідні для обчислень.

Отже, розберемо кожну окремо:

Ф-ія `gcd(a, b)` - простий алгоритм Евкліда.

```
68 def gcd(a: int, b: int) -> int:
69     """ найбільший спільний дільник """
70     logging.debug(f"Обчислення НСД для чисел {a} та {b}")
71     while b:
72         a, b = b, a % b
73     logging.debug(f"Результат НСД: {a}")
74     return a
```

Знаходить найбільший спільний дільник двох чисел.

Використовується для перевірок в RSA.

Ф-ія `extended_gcd(a, b)` - розширений алгоритм Евкліда.

```
76 def extended_gcd(a: int, b: int) -> Tuple[int, int, int]:
77     """ розширений алгоритм Евкліда. повертає (gcd, x, y) такі, що  $a * x + b * y = \text{gcd}$  """
78     logging.debug(f"Обчислення розширеного НСД для чисел {a} та {b}")
79
80     if a == 0:
81         return b, 0, 1
82
83     old_r, r = a, b
84     old_s, s = 1, 0
85     old_t, t = 0, 1
86
87     while r != 0:
88         quotient = old_r // r
89         old_r, r = r, old_r - quotient * r
90         old_s, s = s, old_s - quotient * s
91         old_t, t = t, old_t - quotient * t
92         logging.debug(f"Крок: частка={quotient}, r={r}, s={s}, t={t}")
93
94     logging.debug(f"НСД = {old_r}, x = {old_s}, y = {old_t}")
95     return old_r, old_s, old_t
```

Знаходить НСД і коефіцієнти x, y для рівняння: $ax + by = \text{НСД}(a, b)$.

Потрібна для знаходження мультиплікативного оберненого.

Ф-ія `mod_inverse(a, m)` - знаходження мультиплікативного оберненого за модулем m .

```
97 def mod_inverse(a: int, m: int) -> int:
98     """ знаходження мультиплікативного оберненого за модулем m.  $a * x \equiv 1 \pmod{m}$  """
99     logging.info(f"Обчислення мультиплікативного оберненого для {a} за модулем {m}")
100
101     if m == 1:
102         return 0
103
104     gcd_val, x, _ = extended_gcd(a, m)
105
106     if gcd_val != 1:
107         raise ValueError(f"Мультиплікативне обернене не існує (НСД({a}, {m}) != 1)")
108
109     result = x % m
110     logging.info(f"Результат обчислення мультиплікативного оберненого: {result}")
111     return result
```

Знаходить число x таке, що $(a * x) \bmod m = 1$. Потрібна для обчислення секретного ключа d . Юзає розширений алгоритм Евкліда.

Ф-ія `mod_pow(base, exponent, modulus)` - модульне піднесення до степеня.

```
113 def mod_pow(base: int, exponent: int, modulus: int) -> int:
114     """ Модульне піднесення до степеня (юзається схема Горнера  $\text{base}^{\text{exponent}} \bmod \text{modulus}$ ) """
115     logging.debug(f"Обчислення {base}^{exponent} mod {modulus}")
116
117     if modulus == 1:
118         return 0
119
120     if exponent == 0:
121         return 1
122
123     binary_exp = bin(exponent)[2:]
124     result = 1
125
126     for bit in binary_exp:
127         result = (result * result) % modulus
128         if bit == '1':
129             result = (result * base) % modulus
130
131     return result
```

Швидке піднесення до степеня за модулем. Використовує схему Горнера (бінарний метод). Основна ф-ія для шифрування/розшифрування в RSA.

Ф-ія `bit_length(n)` - повертає довжину числа в бітах.

```

cryptography > lab_4 > _lab_complete > rsapy > ...
133 def bit_length(n: int) -> int:
134     """ повертає довжину числа в бітах """
135     return len(bin(n)) - 2

```

Повертає довжину числа в бітах. Потрібна для перевірки розміру ключів.

Блок №5 | Генерація та перевірка простих чисел

```

137 # генерація та перевірка простих чисел
138 > def decompose_number(n: int) -> Tuple[int, int]: ...
148
149 > def trial_division(n: int) -> bool: ...
159
160 > def miller_rabin_test(n: int, k: int = 40) -> bool: ...
184
185 > def generate_prime_candidate(bit_length: int) -> int: ...
191
192 > def generate_prime(bit_length: int, k: int = 40) -> int: ...
198
199 > def generate_safe_prime(bit_length: int, k: int = 40) -> Tuple[int, int]: ...

```

Цей блок складається з ф-ій, потрібних для генерації та перевірки простих чисел.

Отже, розберемо кожну окремо:

Ф-ія decompose_number(n) - розкладає $n-1$ на добуток

```

138 def decompose_number(n: int) -> Tuple[int, int]:
139     """ розкладає n-1 на добуток (2^s)*d, де d - непарне. повертає (s, d) """
140     s = 0
141     d = n - 1
142
143     while d % 2 == 0:
144         s += 1
145         d //= 2
146
147     return s, d

```

Розкладає $n-1$ на добуток $2^s * d$, де d - непарне. повертає (s, d) .
Потрібно для тесту Міллера-Рабіна.

Ф-ія trial_division(n) - тест пробних ділень на малі прості числа

```

149 def trial_division(n: int) -> bool:
150     """ тест пробних ділень на малі прості числа """
151     if n < 2:
152         return False
153     if n in SMALL_PRIMES:
154         return True
155     if any(n % p == 0 for p in SMALL_PRIMES):
156         return False
157
158     return True

```

Швидка перевірка діленням на малі прості числа, яка відсіює очевидно складені числа.

Ф-ія `miller_rabin_test(n, k=40)` - тест міллера-рабіна з k раундами

```
160 def miller_rabin_test(n: int, k: int = 40) -> bool:
161     """ тест міллера-рабіна з k раундами """
162     if n < 2:
163         return False
164
165     if not trial_division(n):
166         return False
167
168     s, d = decompose_number(n)
169
170     for _ in range(k):
171         a = random.randrange(2, n - 1)
172         x = mod_pow(a, d, n)
173         if x == 1 or x == n - 1:
174             continue
175
176         for _ in range(s - 1):
177             x = mod_pow(x, 2, n)
178             if x == n - 1:
179                 break
180         else:
181             return False
182
183     return True
```

Ймовірнісний тест **Міллера-Рабіна** на простоту. Виконує k раундів перевірки. Точність: ймовірність помилки 4^{-k} .

Ф-ія `generate_prime_candidate(bit_length)` - генерує випадкове непарне число заданої бітової довжини (не обов'язково просте)

```
185 def generate_prime_candidate(bit_length: int) -> int:
186     """ генерує випадкове непарне число заданої бітової довжини (не обов'язково просте) """
187     n = random.getrandbits(bit_length)
188     n |= (1 << bit_length - 1)
189     n |= 1
190     return n
```

Генерує випадкове непарне число заданої довжини. Встановлює старший біт для правильної довжини.

Ф-ія `generate_prime(bit_length, k=40)` - генерує випадкове просте число заданої бітової довжини (точно просте)

```
192 def generate_prime(bit_length: int, k: int = 40) -> int:
193     """ генерує випадкове просте число заданої бітової довжини (точно просте) """
194     while True:
195         n = generate_prime_candidate(bit_length)
196         if miller_rabin_test(n, k):
197             return n
```

Генерує просте число заданої довжини. Використовує тест Міллера-Рабіна для перевірки.

Ф-ія `generate_safe_prime(bit_length, k=40)` - генерує безпечне просте число p

```
199 def generate_safe_prime(bit_length: int, k: int = 40) -> Tuple[int, int]:
200     """ генерує безпечне просте число p таке, що q = (p-1)/2 також просте. повертає пару (p, q) """
201     logging.info(f"Генерація {bit_length}-бітного безпечного простого числа...")
202
203     while True:
204         q = generate_prime(bit_length - 1, k)
205
206         p = 2 * q + 1
207
208         if miller_rabin_test(p, k):
209             logging.info(f"Знайдено безпечне просте p = {p}")
210             logging.info(f"Перевірка: q = (p-1)/2 = {q} - також просте число")
211             logging.info(f"Перевірка: p = 2q + 1 = {p}")
212             return p, q
```

Генерує "безпечне" просте число p таке, що $q = (p-1)/2$ також просте. повертає пару (p, q) . Забезпечує, що $(p-1)/2$ теж просте. Важливо для криптостійкості RSA.

Блок №6 | Клас RSAMessage

```
214 # розбиття повідомлень на блоки, підготовки їх до RSA та відновлення оригінального повідомлення з блоків
215 class RSAMessage:
216     """ клас для роботи з повідомленнями RSA """
217     def __init__(self, content: Union[str, int], public_key: RSAPublicKey = None): ...
218
219     @staticmethod
220     def calculate_block_size(n: int) -> int: ...
221
222     @staticmethod
223     def from_blocks(blocks: List[int], is_text: bool, block_size: int, original_bytes_length: int = 0) -> 'RSAMessage': ...
224
225     def _prepare_blocks(self, content: Union[str, int], block_size: int): ...
226
227     def _prepare_text_blocks(self, text: str, block_size: int): ...
228
229     def _prepare_number_blocks(self, number: int, block_size: int): ...
230
231     def _blocks_to_number(self, blocks: List[int]) -> int: ...
232
233     def _blocks_to_text(self, blocks: List[int], block_size: int) -> str: ...
```

Цей блок включає клас **RSAMessage** - відповідає за підготовку повідомлень до RSA шифрування. Основна ідея - розбиття великих повідомлень на менші блоки, бо RSA може шифрувати тільки числа, менші за модуль n .

Отже, розберемо кожну окремо:

Ф-ія `__init__(self, content, public_key)` - ініт клас RSAMessage

```
215 class RSAMessage:
216     """ клас для роботи з повідомленнями RSA """
217     def __init__(self, content: Union[str, int], public_key: RSAPublicKey = None):
218         self.is_text = isinstance(content, str)
219         self.original_content = content
220         self.blocks: List[int] = []
221         self.original_bytes_length = 0
222
223         if public_key:
224             self.block_size = self.calculate_block_size(public_key.n)
225             self._prepare_blocks(content, self.block_size)
```

Ініціалізує об'єкт повідомлення. Визначає тип (текст/число). Розбиває на блоки, якщо наданий публічний ключ.

Ф-ія `calculate_block_size(n)` - обчислює безпечний розмір блоку для модуля n

```
227 @staticmethod
228 def calculate_block_size(n: int) -> int:
229     # розмір блоку в байтах - це просто к-сть байт в n - 1
230     # (мінус 1 для гарантії, що блок буде менше n)
231     return (n.bit_length() - 1) // 8
```

Розраховує розмір блоку для модуля n (мінус 1 для гарантії, що блок буде менше n)

Ф-ія from_blocks(blocks, is_text, block_size, original_bytes_length) - створює повідомлення RSA з блоків

```
238 @staticmethod
239 def from_blocks(blocks: List[int], is_text: bool, block_size: int, original_bytes_length: int = 0) -> 'RSAMessage':
240     message = RSAMessage._new_(RSAMessage)
241     message.blocks = blocks
242     message.is_text = is_text
243     message.block_size = block_size
244     message.original_bytes_length = original_bytes_length
245
246     if is_text:
247         message.original_content = message._blocks_to_text(blocks, block_size)
248     else:
249         message.original_content = blocks[0]
250
251     return message
```

Створює повідомлення з готових блоків. Відновлює оригінальний текст/число.

Ф-ія _prepare_blocks(self, content, block_size) - підготовка блоків залежно від типу вмісту (текст/число)

```
253 def _prepare_blocks(self, content: Union[str, int], block_size: int):
254     if self.is_text:
255         self._prepare_text_blocks(content, block_size)
256     else:
257         self.blocks = [content]
```

Обирає тип ф-ії обробки блоків залежно від типу вмісту (текст/число)

Ф-ія _prepare_text_blocks(self, text, block_size) - розбиває текстове повідомлення на блоки (якщо велике)

```
215 class RSAMessage:
257     def _prepare_text_blocks(self, text: str, block_size: int):
258         """ розбиває текстове повідомлення на блоки (якщо велике) """
259         logging.info(f"Підготовка текстового повідомлення: {len(text)} символів")
260         bytes_data = text.encode('utf-8')
261
262         for i in range(0, len(bytes_data), block_size):
263             block = bytes_data[i:i + block_size]
264             padded_block = block.ljust(block_size, b'\x00')
265             number = int.from_bytes(padded_block, 'big')
266             self.blocks.append(number)
267             logging.debug(f"Блок {i//block_size}: {len(block)} байт -> {number}")
268
269         logging.info(f"Створено {len(self.blocks)} блоків")
```

Конвертує текст в UTF-8. Розбиває на блоки. Доповнює нулями до потрібного розміру.

Ф-ія _prepare_number_blocks(self, number, block_size) -

```
270 def _prepare_number_blocks(self, number: int, block_size: int):
271     """ розділяє число на блоки відповідно до розміру модуля """
272     max_block = (1 << (block_size * 8)) - 1 # макс значення блоку
273     number_str = str(number) # в рядок
274     block_size_dec = len(str(max_block)) # розмір блоку в десяткових цифрах
275
276     # розділяємо на блоки з кінця
277     blocks = []
278     while number_str:
279         block = number_str[-block_size_dec:] if len(number_str) > block_size_dec else number_str
280         number_str = number_str[:-block_size_dec] if len(number_str) > block_size_dec else ""
281         blocks.insert(0, int(block))
282
283     self.blocks = blocks
```

Перетворює число в байти. Розбиває на блоки потрібного розміру.

Ф-ія `_blocks_to_number(self, blocks)` - конвертує блоки назад у число

```
285     def _blocks_to_number(self, blocks: List[int]) -> int:
286         """Об'єднує блоки назад в число конкатенацією"""
287         return int(''.join(str(block) for block in blocks))
```

Збирає блоки назад в оригінальне число.

Ф-ія `_blocks_to_text(self, blocks, block_size)` - конвертує блоки назад у текст

```
215     class RSAMessage:
216
217     def _blocks_to_text(self, blocks: List[int], block_size: int) -> str:
218         """Конвертує блоки назад у текст """
219         text_parts = []
220
221         for i, number in enumerate(blocks):
222             try:
223                 block_bytes = number.to_bytes(block_size, 'big')
224                 if 1 == len(blocks) - 1:
225                     block_bytes = block_bytes.rstrip(b'\x00')
226                 text_part = block_bytes.decode('utf-8')
227                 text_parts.append(text_part)
228             except (ValueError, UnicodeDecodeError) as e:
229                 logging.error(f"Не вдалося декодувати блок {i}: {number}")
230                 raise ValueError(f"Не вдалося декодувати блок {i}: {str(e)}")
231
232         result = ''.join(text_parts)
233         logging.info(f"Успішно декодований текст: {len(result)} символів")
234         return result
```

Збирає блоки назад в оригінальне повідомлення, видаляючи доповнюючі нулі.

Отже, в цілому:

- **Текст** - конвертується в UTF-8 байти, потім в числа
- **Числа** - розбиваються на блоки відповідного розміру
- Кожен блок доповнюється **нулями** до потрібної довжини

Основні методи:

- **`_prepare_text_blocks`** - розбиває текст на блоки
- **`_prepare_number_blocks`** - розбиває число на блоки
- **`_blocks_to_text/_blocks_to_number`** - збирає блоки назад

Блок №7 | Генерація ключа

```
321 # генерація ключа
322 > def validate_prime_pair(p: int, q: int, bit_length: int) -> bool: ...
349
350 > def generate_key_pair(bit_length: int = MIN_BIT_LENGTH) -> Tuple[RSAPublicKey, RSAPrivateKey]: ...
```

Цей блок складається з ф-ій, потрібних для генерації RSA ключа.

Отже, розберемо кожну окремо:

Ф-ія `validate_prime_pair(p, q, bit_length)` - перевіряє пару простих чисел на відповідність вимогам RSA

```
322 def validate_prime_pair(p: int, q: int, bit_length: int) -> bool:
323     """ перевіряє пару простих чисел чи відповідають вимогам RSA """
324
325     logging.debug(f"Перевірка пари простих чисел: p={p}, q={q}")
326
327     # перевірка бітової довжини
328     if p.bit_length() < bit_length or q.bit_length() < bit_length:
329         logging.warning("Прості числа занадто малі")
330         return False
331
332     # перевірка простоти
333     if not (miller_rabin_test(p) and miller_rabin_test(q)):
334         logging.warning("Числа не є простими")
335         return False
336
337     # p і q повинні бути різними
338     if p == q:
339         logging.warning("Прості числа рівні")
340         return False
341
342     # перевірка, що p-1 і q-1 мають великі прості множники
343     if gcd(p - 1, DEFAULT_E) == 1 and gcd(q - 1, DEFAULT_E) == 1:
344         logging.debug("Пара простих чисел успішно перевірена")
345         return True
346
347     logging.warning("Неналежна пара простих чисел для RSA")
348     return False
```

Перевіряє чи підходить пара простих чисел для RSA:

1. Достатня довжина в бітах
2. Обидва числа прості
3. Числа різні
4. $p-1$ і $q-1$ взаємно прості з e (для безпеки)

Якщо все **True** - вважається підходящою.

Ф-ія `generate_key_pair(bit_length)` - генерує пару ключів RSA

```
350 def generate_key_pair(bit_length: int = MIN_BIT_LENGTH) -> Tuple[RSAPublicKey, RSAPrivateKey]:
351     """ генерує пару ключів RSA. повертає кортеж (public_key, private_key) """
352     if bit_length < MIN_BIT_LENGTH:
353         raise ValueError(f"Довжина ключа має бути не менше {MIN_BIT_LENGTH} біт")
354
355     logging.info(f"Генерація {bit_length}-бітної пари ключів RSA")
356
357     p_q_length = bit_length // 2
358
359     while True:
360         logging.info(f"Генерація першого простого числа (p) довжиною {p_q_length} біт...")
361         p, _ = generate_safe_prime(p_q_length)
362         logging.info(f"Згенеровано p: {p}\n")
363
364         logging.info(f"Генерація другого простого числа (q) довжиною {p_q_length} біт...")
365         q, _ = generate_safe_prime(p_q_length)
366         logging.info(f"Згенеровано q: {q}\n")
367
368         if not validate_prime_pair(p, q, p_q_length):
369             logging.info("Згенеровані прості числа не пройшли валідацію, повторна спроба...")
370             continue
371
372         n = p * q
373
374         if n.bit_length() != bit_length:
375             logging.info(f"Згенерований ключ має неправильну довжину ({n.bit_length()} біт), повторна спроба...\n")
376             continue
```

```

350 def generate_key_pair(bit_length: int = MIN_BIT_LENGTH) -> Tuple[RSAPublicKey, RSAPrivateKey]:
351     logging.info(f"Обчислено n = p*q: {n}")
352
353     phi = (p - 1) * (q - 1)
354     logging.info(f"Обчислено φ(n) = (p-1)(q-1): {phi}")
355
356     e = DEFAULT_E
357     logging.info(f"Використовується e = {e}")
358
359     try:
360         d = mod_inverse(e, phi)
361         logging.info(f"Обчислено d = e^(-1) mod φ(n): {d}")
362
363         public_key = RSAPublicKey(e=e, n=n)
364         private_key = RSAPrivateKey(d=d, p=p, q=q, n=n)
365
366         logging.info("Пара ключів успішно згенерована")
367         return public_key, private_key
368
369     except ValueError as err:
370         logging.warning(f"Помилка обчислення d: {err}, повторна спроба з новими простими числами...")
371         continue

```

Генерує пару ключів RSA:

- Перевіряє мінімальну довжину (256 біт)
- Генерує безпечні прості p і q довжиною $\text{bit_length}/2$
- Перевіряє їх через `validate_prime_pair`

Обчислює:

- ✧ $n = p \cdot q$ (модуль)
- ✧ $\phi(n) = (p-1)(q-1)$ (ф-ія Ейлера - к-сть чисел, взаємно простих з n)
- ✧ $e = 65537$ (стандартне)
- ✧ $d = e^{-1} \bmod \phi(n)$ (секретний ключ)
- ✧ Повертає пару (публічний, приватний) ключів

Блок №8 | Шифрування/Розшифрування

```

480 # шифрування/розшифрування
481 > def encrypt_block(block: int, public_key: RSAPublicKey) -> int: ...
482
483 > def decrypt_block(block: int, private_key: RSAPrivateKey) -> int: ...
484
485 > def encrypt(message: Union[str, int, RSAMessage], public_key: RSAPublicKey) -> Tuple[List[int], bool, int]: ...
486
487 > def decrypt(encrypted_blocks: List[int], private_key: RSAPrivateKey, is_text: bool = False, original_bytes_length: int = 0) -> Union[str, int]: ...

```

Цей блок складається з ф-ій, потрібних для шифрування/розшифрування.

Отже, розберемо кожну окремо:

Ф-ія `encrypt_block(block, public_key)` - шифрує один блок

```

481 def encrypt_block(block: int, public_key: RSAPublicKey) -> int:
482     """ шифрує один блок """
483     logging.debug(f"Шифрування блоку: {block}")
484     if block >= public_key.n:
485         raise ValueError("Блок завеликий для даного ключа")
486     ciphertext = mod_pow(block, public_key.e, public_key.n)
487     logging.debug(f"Зашифрований блок: {ciphertext}")
488     return ciphertext

```

Шифрує один блок за формулою $C = M^e \bmod n$. Перевіряє чи блок не більший за модуль. Юзає швидко модульне піднесення до степеня.

Ф-ія `decrypt_block(block, private_key)` - розшифровує один блок

```
410 def decrypt_block(block: int, private_key: RSAPrivateKey) -> int:
411     """ розшифровує один блок """
412     logging.debug(f"Розшифрування блоку: {block}")
413     plaintext = mod_pow(block, private_key.d, private_key.n)
414     logging.debug(f"Розшифрований блок: {plaintext}")
415     return plaintext
```

Розшифровує блок за формулою $M = C^d \bmod n$. Використовує приватний ключ (d, n) .

Ф-ія `encrypt(message, public_key)` - шифрує повідомлення (текст/число)

```
417 def encrypt(message: Union[str, int, RSAMessage], public_key: RSAPublicKey) -> Tuple[List[int], bool, int]:
418     """ шифрує повідомлення (текст/число). повертає: (зашифровані_блоки, чи_це_текст, довжина_оригінальних_байтів) """
419     if not isinstance(message, RSAMessage):
420         message = RSAMessage(message, public_key)
421
422     encrypted_blocks = []
423     logging.info(f"Шифрування {'текстового' if message.is_text else 'числового'} повідомлення з {len(message.blocks)} блоками")
424
425     for i, block in enumerate(message.blocks):
426         try:
427             encrypted_block = encrypt_block(block, public_key)
428             encrypted_blocks.append(encrypted_block)
429         except ValueError as e:
430             logging.error(f"Помилка шифрування блоку {i + 1}: {str(e)}")
431             raise
432
433     logging.info("Шифрування завершено")
434     return encrypted_blocks, message.is_text, message.original_bytes_length
```

Створює `RSAMessage` якщо потрібно. Шифрує кожен блок окремо. Повертає (зашифровані_блоки, чи_це_текст, довжина_байтів).

Ф-ія `decrypt(encrypted_blocks, private_key, is_text, original_bytes_length)` - розшифровує повідомлення

```
436 def decrypt(encrypted_blocks: List[int], private_key: RSAPrivateKey, is_text: bool = False, original_bytes_length: int = 0) -> Union[str, int]:
437     """ розшифровує повідомлення """
438     block_size = RSAMessage.calculate_block_size(private_key.n)
439     decrypted_blocks = []
440
441     logging.info(f"Розшифрування {len(encrypted_blocks)} блоку")
442     print("\nДані для перевірки в RSA калькуляторі:")
443     print("-----")
444
445     for i, block in enumerate(encrypted_blocks):
446         print(f"Блок {i + 1}:")
447         print(f"C (зашифроване повідомлення) = {block}")
448         print(f"N (модуль) = {private_key.n}")
449         print(f"D (секретний ключ) = {private_key.d}")
450         print(f"E (відкритий ключ) = {DEFAULT_E}")
451
452         decrypted_block = decrypt_block(block, private_key)
453         decrypted_blocks.append(decrypted_block)
454
455         print(f"Розшифроване значення = {decrypted_block}")
456         print("-----")
457
458     message = RSAMessage.from_blocks(
459         decrypted_blocks,
460         is_text,
461         block_size,
462         original_bytes_length
463     )
464     return message.original_content
```

Розшифровує всі блоки. Виводить дані для перевірки (C, N, D, E) (для <https://www.dcode.fr/rsa-cipher>). Збирає блоки в оригінальне повідомлення через `RSAMessage`. Повертає текст або число (в залежності від типу).

Блок №9 | Підпис

```
466 # підпис
467 > def sign(message: Union[str, int, RSAMessage], private_key: RSAPrivateKey) -> List[int]: ...
481
482 > def verify(message: Union[str, int, RSAMessage], signatures: List[int], public_key: RSAPublicKey) -> bool: ...
```

Цей блок складається з ф-ій, потрібних створення цифрового RSA підпису.

Отже, розберемо кожну окремо:

Ф-ія sign(message, private_key) - створення підпису

```
467 def sign(message: Union[str, int, RSAMessage], private_key: RSAPrivateKey) -> List[int]:
468     """ створення підпису """
469     if not isinstance(message, RSAMessage):
470         message = RSAMessage(message, RSAPublicKey(private_key.d, private_key.n))
471
472     signatures = []
473     logging.info(f"Підписання повідомлення з {len(message.blocks)} блоками")
474
475     for i, block in enumerate(message.blocks):
476         signature = mod_pow(block, private_key.d, private_key.n)
477         signatures.append(signature)
478         logging.info(f"Блок {i + 1} підписано: {signature}")
479
480     return signatures
```

Створює цифровий підпис за формулою $S = M^d \bmod n$. Розбиває повідомлення на блоки якщо потрібно. Підписує кожен блок окремо приватним ключем. Повертає список підписів.

Ф-ія verify(message, signatures, public_key) - перевірка підпису

```
482 def verify(message: Union[str, int, RSAMessage], signatures: List[int], public_key: RSAPublicKey) -> bool:
483     """ перевірка підпису """
484     if not isinstance(message, RSAMessage):
485         message = RSAMessage(message, public_key)
486
487     if len(signatures) != len(message.blocks):
488         logging.error("Кількість підписів не відповідає кількості блоків повідомлення")
489         return False
490
491     logging.info(f"Перевірка {len(signatures)} підписів")
492
493     for i, (block, signature) in enumerate(zip(message.blocks, signatures)):
494         computed = mod_pow(signature, public_key.e, public_key.n)
495         if computed != block:
496             logging.error(f"Перевірка підпису для блоку {i + 1} не вдалася")
497             return False
498         logging.info(f"Підпис блоку {i + 1} перевірено")
499
500     logging.info(f"Всі підписи успішно перевірено")
501     return True
```

Важливо: підпис робиться приватним ключем, а перевірка - публічним, що гарантує автентичність відправника.

Узагальнення

RSA криптосистема, реалізована в даному коді, представляє собою повний набір криптографічних операцій. В основі лежить робота з великими простими числами та модульною арифметикою. Код організований навколо двох ключових класів: для роботи з ключами (публічним та приватним) та для обробки повідомлень.

Математична основа - включає реалізацію алгоритма Евкліда для пошуку НСД, модульну арифметику та тест Міллера-Рабіна для генерації надійних простих чисел. Також задля безпеки використовуються тільки "безпечні" прості числа, довжиною не менше **256 біт**, а відкрита експонента = 65537.

Для роботи з повідомленнями реалізовано механізм **розбиття на блоки** через клас **RSAMessage**. Це дозволяє обробляти як текстові, так і числові дані **будь-якого розміру**. Основні криптографічні операції включають **шифрування** та **розшифрування**, а також створення та перевірку підпису.

TEST.py | Покроковий розбір та пояснення

Загальна структура test.py

```
cryptography > lab_4 > _lab_complete > test.py > ...
1  from rsa import *
2
3  > def print_section(title: str):...
7
8  > def format_key_info(name: str, public_key: RSAPublicKey, private_key: RSAPrivateKey) -> str:...
25
26 > def demonstrate_key_generation(alice_bit_length, bob_bit_length):...
40
41 > def demonstrate_encryption(alice_keys: Tuple[RSAPublicKey, RSAPrivateKey], bob_keys: Tuple[RSAPublicKey, RSAPrivateKey]):...
95
96 > def demonstrate_signature(alice_keys: Tuple[RSAPublicKey, RSAPrivateKey], bob_keys: Tuple[RSAPublicKey, RSAPrivateKey]):...
150
151 > def demonstrate_key_exchange(alice_keys: Tuple[RSAPublicKey, RSAPrivateKey], bob_keys: Tuple[RSAPublicKey, RSAPrivateKey]):...
227
228 def main():
229     print("\nДЕМОНСТРАЦІЯ КРИПТОСИСТЕМИ RSA\n")
230     print("Цей скрипт демонструє роботу RSA на практичних прикладах")
231
232     # генерація ключів
233     alice_keys, bob_keys = demonstrate_key_generation(512, 512)
234
235     # приклад шифрування текст повідомлень
236     demonstrate_encryption(alice_keys, bob_keys)
237
238     # приклад підпису
239     demonstrate_signature(alice_keys, bob_keys)
240
241     # приклад протоколу обміну ключами
242     demonstrate_key_exchange(alice_keys, bob_keys)
243
244     print("\nДЕМОНСТРАЦІЯ ЗАВЕРШЕНА")
245
246 if __name__ == "__main__":
247     main()
```

Ф-ія main() - основна, звідки викликаються вже всі необхідні тести.

Блок №0 | Допоміжні/візуальні функції

```
3 > def print_section(title: str): ...
7
8 > def format_key_info(name: str, public_key: RSAPublicKey, private_key: RSAPrivateKey) -> str: ...
```

Цей блок складається з ф-ій, потрібних форматування виводу.

Отже, розберемо кожну окремо:

Ф-ія print_section(title: str) - вивід секції форматовано

```
3 def print_section(title: str):
4     print(f"\n{'='*60}")
5     print(f"{title}")
6     print(f"\n{'='*60}\n")
```

Друкує заголовок секції, використовуючи “=”.

Ф-ія format_key_info(name, public_key, private_key) - вивід інформацію про ключі форматовано

```
8 def format_key_info(name: str, public_key: RSAPublicKey, private_key: RSAPrivateKey) -> str:
9     return f"""
10 Ключі для {name}:
11 -----
12 Відкритий ключ (e, n):
13     e = {public_key.e}
14     n = {public_key.n}
15     Довжина ключа: {public_key.n.bit_length()} bit
16
17 Закритий ключ:
18     d = {private_key.d}
19     p = {private_key.p}
20     q = {private_key.q}
21
22 Перевірка:
23     e*d mod φ(n) = {(public_key.e * private_key.d) % ((private_key.p - 1) * (private_key.q - 1))} (має бути 1)
24     """
```

Форматує інформацію про пару ключів RSA. Відображає відкритий ключ (**e**, **n**), закритий ключ (**d**, **p**, **q**), перевірку коректності ключів через **e*d mod φ(n)** (результат має бути 1).

Блок/тест №1 | Генерація ключів

```
26 def demonstrate_key_generation(alice_bit_length, bob_bit_length):
27     print_section("ГЕНЕРАЦІЯ КЛЮЧІВ RSA")
28
29     # ключі для аліси
30     print(f"1. Генерація ключів для Аліси ({alice_bit_length} bit)...")
31     alice_public, alice_private = generate_key_pair(alice_bit_length)
32     print(format_key_info("Аліси", alice_public, alice_private))
33
34     # ключі для боба
35     print(f"2. Генерація ключів для Боба ({bob_bit_length} bit)...")
36     bob_public, bob_private = generate_key_pair(bob_bit_length)
37     print(format_key_info("Боба", bob_public, bob_private))
38
39     return (alice_public, alice_private), (bob_public, bob_private)
```

Ф-ія demonstrate_key_generation демонструє процес генерації ключів RSA для двох користувачів (Аліси і Боба):

- Генерує ключі заданої бітової довжини
- Виводить повну інформацію про ключі
- Робить перевірку правильності ключів та повертає пари ключів

Вивід тесту №1 | Генерація ключів:

```
ДЕМОНСТРАЦІЯ КРИПТОСИСТЕМИ RSA

Цей скрипт демонструє роботу RSA на практичних прикладах

=====
ГЕНЕРАЦІЯ КЛЮЧІВ RSA
=====
```

Для Аліси:

```
1. Генерація ключів для Аліси (512 біт)...
[x] Генерація 512-бітної пари ключів RSA
[x] Генерація першого простого числа (p) довжиною 256 біт...
[x] Генерація 256-бітного безпечного простого числа...
[i] Знайдено безпечне просте p = 76370951970425063299566251133867171875990323891930043883836489381359217127227
[v] Перевірка: q = (p-1)/2 = 38185475985212531649783125566933585937995161945965021941918244690679608563613 - також просте число
[v] Перевірка: p = 2q + 1 = 76370951970425063299566251133867171875990323891930043883836489381359217127227
[x] Згенеровано p: 76370951970425063299566251133867171875990323891930043883836489381359217127227

[x] Генерація другого простого числа (q) довжиною 256 біт...
[x] Генерація 256-бітного безпечного простого числа...
[i] Знайдено безпечне просте p = 113464320461159730980604381224564850177744016847715003595674929410878142111679
[v] Перевірка: q = (p-1)/2 = 56732160230579865490302190612282425088872008423857501797837464705439071055839 - також просте число
[v] Перевірка: p = 2q + 1 = 113464320461159730980604381224564850177744016847715003595674929410878142111679
[x] Згенеровано q: 113464320461159730980604381224564850177744016847715003595674929410878142111679

[i] Обчислено n = p*q: 866537816829614758365612125680398008165915076604119432213102574382502074727541767306183136520669113134029351292040481463349232156107057313
423381785584133
[i] Обчислено φ(n) = (p-1)(q-1): 86653781682961475836561212568039800816591507660411943221310257438250207472735193203375155172638894250167091926998674712260958710
8627545894631144426345228
[i] Використовується e = 65537
[>] Обчислення мультиплікативного оберненого для 65537 за модулем 86653781682961475836561212568039800816591507660411943221310257438250207472735193203375155172638
8942501670919269986747122609587108627545894631144426345228
[>] Результат обчислення мультиплікативного оберненого: 3647320700862859622734548497321174123399862175383747112891930276246110132159757130249340625209766070752530
514183782968695197910499282708294603050475315213
[i] Обчислено d = e^(-1) mod φ(n): 3647320700862859622734548497321174123399862175383747112891930276246110132159757130249340625209766070752530514183782968695197910
499282708294603050475315213
[=] Пара ключів успішно згенерована

Ключі для Аліси:
-----
Відкритий ключ (e, n):
e = 65537
n = 866537816829614758365612125680398008165915076604119432213102574382502074727541767306183136520669113134029351292040481463349232156107057313423381785584133
Довжина ключа: 512 біт
Закритий ключ:
d = 3647320700862859622734548497321174123399862175383747112891930276246110132159757130249340625209766070752530514183782968695197910499282708294603050475315213
p = 76370951970425063299566251133867171875990323891930043883836489381359217127227
q = 113464320461159730980604381224564850177744016847715003595674929410878142111679
Перевірка:
e * d mod φ(n) = 1 (має бути 1)
```

Для Боба:

```
2. Генерація ключів для Боба (512 біт)...
[x] Генерація 512-бітної пари ключів RSA
[x] Генерація першого простого числа (p) довжиною 256 біт...
[x] Генерація 256-бітного безпечного простого числа...
[i] Знайдено безпечне просте p = 84400443802613178358536425976923681902478757765686200069429346686343920251959
[v] Перевірка: q = (p-1)/2 = 422002219013858917926821298846184095123937888284310003471467313171960125979 - також просте число
[v] Перевірка: p = 2q + 1 = 84400443802613178358536425976923681902478757765686200069429346686343920251959
[x] Згенеровано p: 84400443802613178358536425976923681902478757765686200069429346686343920251959

[x] Генерація другого простого числа (q) довжиною 256 біт...
[x] Генерація 256-бітного безпечного простого числа...
[i] Знайдено безпечне просте p = 83946732795587631372033027864416104031536668544658884803491738273966414720459
[v] Перевірка: q = (p-1)/2 = 41973366397793815686016513932208052015768334272329442401745869136983207360229 - також просте число
[v] Перевірка: p = 2q + 1 = 83946732795587631372033027864416104031536668544658884803491738273966414720459
[x] Згенеровано q: 83946732795587631372033027864416104031536668544658884803491738273966414720459

[i] Обчислено n = p*q: 7085141503726978555001660104357788344512942489364269099941975374537018133676133162438797866991606868737799549992835237860628256577359101169
838145832129181
[i] Обчислено φ(n) = (p-1)(q-1): 7085141503726978555001660104357788344512942489364269099941975374537018133675964815262199666181876299283958210206901222434317911492486180084877835497156764
2486180084877835497156764
[i] Використовується e = 65537
[>] Обчислення мультиплікативного оберненого для 65537 за модулем 708514150372697855500166010435778834451294248936426909994197537453701813367596481526219966618187
6299283958210206901222434317911492486180084877835497156764
[>] Результат обчислення мультиплікативного оберненого: 5427073309536511031342346113474235544723586874072452337108612902661981938607709137222674569210220031799665
8704607540986954355426235989782910549360202217
[i] Обчислено d = e^(-1) mod φ(n): 5427073309536511031342346113474235544723586874072452337108612902661981938607709137222674569210220031799665870460754098695435542
6235989782910549360202217
[=] Пара ключів успішно згенерована

Ключі для Боба:
-----
Відкритий ключ (e, n):
e = 65537
n = 7085141503726978555001660104357788344512942489364269099941975374537018133676133162438797866991606868737799549992835237860628256577359101169838145832129181
Довжина ключа: 512 біт
Закритий ключ:
d = 54270733095365110313423461134742355447235868740724523371086129026619819386077091372226745692102200317996658704607540986954355426235989782910549360202217
p = 84400443802613178358536425976923681902478757765686200069429346686343920251959
q = 83946732795587631372033027864416104031536668544658884803491738273966414720459
Перевірка:
e * d mod φ(n) = 1 (має бути 1)
```

Пояснення виводу тесту №1:

Вивід показує, що генерація ключів пройшла успішно для обох користувачів. Ключові моменти:

1. Довжина ключів: **512 біт** (по **256 біт** на кожне просте число)
2. Юзають безпечні прості числа (**Safe Primes**)
3. Публічна експонента **e = 65537** для обох користувачів
4. Перевірка **$e * d \bmod \phi(n) = 1$** пройшла успішно для обох пар ключів
5. Модулі **n** мають коректну довжину (**512 біт**)

Всі параметри відповідають вимогам безпеки та теорії RSA.

Блок/тест №2 | Шифрування/Дешифрування

[illegible]

Ф-ія demonstrate_encryption показує весь процес обміну повідомленнями між **Алісою** та **Бобом**. Для **5** різних типів повідомлень (**числа різної довжини, текст з емодзі, кирилицею та спецсимволами**) відбувається:

Боб -> Аліса:

- Шифрування повідомлення публічним ключем Аліси
- Розшифрування Алісою своїм приватним ключем
- Перевірка співпадіння з оригіналом

Аліса -> Боб (відповідь):

- Створення відповіді (для чисел *2, для тексту - "Re: ...")
- Шифрування відповіді публічним ключем Боба
- Розшифрування Бобом своїм приватним ключем
- Перевірка коректності

Вивід тесту №2 | Шифрування/Дешифрування:

Тест №1.1 - Боб Алісі

```
=====
ШИФРУВАННЯ ПОВІДОМЛЕНЬ
=====

Тест 1.1 - Боб надсилає Алісі:
Опис: Просте числове повідомлення
Оригінальне повідомлення: 12345

Боб шифрує повідомлення для Аліси:
[+] Шифрування числового повідомлення з 1 блоками
[+] Шифрування завершено
Зашифровані блоки числа: [4849539510750868945840891740081989733005250978498681600190837844180216340161584714925080303961139617742395624865261502483062990479188067369685937306519176]

Аліса розшифровує повідомлення:
[-] Розшифрування 1 блоку (-iv)

Дані для перевірки в RSA калькуляторі:
-----

Блок 1:
С (зашифроване повідомлення) = 4849539510750868945840891740081989733005250978498681600190837844180216340161584714925080303961139617742395624865261502483062990479188067369685937306519176
N (модуль) = 8665378168296147583656121256803980008165915076604119432213102574382502074727541767306183136520669113134029351292040481463349232156107057313423381785584133
D (секретний ключ) = 3647320708062859622734548497321174123399862175383747112891930276246110132159757130249340625209766070752530514183782968695197910499282708294603050475315213
E (відкритий ключ) = 65537
Розшифроване значення = 12345
-----
Розшифроване повідомлення: 12345
Перевірка успішна: True
```

Тест №1.2 - Аліса Бобу

```
Тест 1.2 - Аліса відповідає Бобу:
Оригінальне повідомлення: 24690

Аліса шифрує повідомлення для Боба:
[+] Шифрування числового повідомлення з 1 блоками
[+] Шифрування завершено
Зашифроване число: 6398373433171126031440185509023459456314494157824786783544791591779573565403974960573124910983516442159103544479853202899691335745922738281385516841014586

Боб розшифровує повідомлення:
[-] Розшифрування 1 блоку (-iv)

Дані для перевірки в RSA калькуляторі:
-----

Блок 1:
С (зашифроване повідомлення) = 6398373433171126031440185509023459456314494157824786783544791591779573565403974960573124910983516442159103544479853202899691335745922738281385516841014586
N (модуль) = 7085141503726978550016601043577883445129424893642690999419753745370818133676133162438797866991606868737799549992835237860628256577359101169838145832129181
D (секретний ключ) = 5427073309536511031342346113474235544723586874072452337108612902661981938607709137226745692102200317996658704607540986954355426235989782910549360202217
E (відкритий ключ) = 65537
Розшифроване значення = 24690
-----
Розшифроване повідомлення: 24690
Перевірка успішна: True

=====
```

[illegible]

Тест №3.1 - Боб Алісі

```
Тест 3.1 - Боб надсилає Алісі:
Опис: Простий тест з емоді!
Оригінальне повідомлення: Привіт, RSA! 🍌

Боб шифрує повідомлення для Аліси:
[>] Підготовка текстового повідомлення: 14 символів
[i] Створено 1 блоків
[+] Шифрування текстового повідомлення з 1 блоками
[+] Шифрування завершено
Зашифровані блоки: [8396049485479162109105483136408939193748430551172047477893068142766387617686041581520505051377940384984016367039224115775947949542011405727914700048224349]

Аліса розшифровує повідомлення:
[-] Розшифрування 1 блоку (-ів)

Дані для перевірки в RSA калькуляторі:
-----

Блок 1:
С (зашифроване повідомлення) = 8396049485479162109105483136408939193748430551172047477893068142766387617686041581520505051377940384984016367039224115775947949542011405727914700048224349
N (модуль) = 8665378168296147583656121256803980008165915076604119432213102574382502074727541767306183136520669113134029351292040481463349232156107057313423381785584133
D (секретний ключ) = 364732070086285962273454849732117412339986217538374711289193027624611013215975713024934062520976607052530514183782968695197910499282708294603050475315213
Е (відкритий ключ) = 65537
Розшифроване значення = 42681799571905479707929380822325560275158905513554347035003004743614997299361263315090325597680806887316036918387563013979749200248715730694395861663744
-----
[=] Успішно декодований текст: 14 символів
Розшифроване повідомлення: Привіт, RSA! 🍌
Перевірка успішна: True
```

Тест №3.2 - Аліса Бобу

```
Тест 3.2 - Аліса відповідає Бобу:
Оригінальне повідомлення: Re: Привіт, RSA! 🍌... Отримано!

Аліса шифрує повідомлення для Боба:
[>] Підготовка текстового повідомлення: 31 символів
[i] Створено 1 блоків
[+] Шифрування текстового повідомлення з 1 блоками
[+] Шифрування завершено
Зашифровані блоки: [108058795803335237326746257496552861426124274594124591980025019841281347965137142464021791812578401595597321583221974021536948611197500951750513472119387513472119387]

Боб розшифровує повідомлення:
[-] Розшифрування 1 блоку (-ів)

Дані для перевірки в RSA калькуляторі:
-----

Блок 1:
С (зашифроване повідомлення) = 1080587958033352373267462574965528614261242745941245919800250198412813479651371424640217918125784015955973215832219740215369486111977500951750513472119387
N (модуль) = 7085141503726978555001660104357788344512942489364269099941975374537018133676133162438797866991606868737799549992835237860628256577359101169838145832129181
D (секретний ключ) = 54270733095365110313423461134742355447235868740724523371086129026619819386077091372226745692102200317996658704607540986954355426235989782910549360202217
Е (відкритий ключ) = 65537
Розшифроване значення = 1685702425755681536263285852307385556181850704890349401766000336431331829318381251759811335785922737955780712026987503278040701387979272049867268030464
-----
[=] Успішно декодований текст: 31 символів
Розшифроване повідомлення: Re: Привіт, RSA! 🍌... Отримано!
Перевірка успішна: True

=====
```

Тест №4.1 - Боб Алісі

```
Тест 4.1 - Боб надсилає Алісі:
Опис: Довге повідомлення з кирилицею
Оригінальне повідомлення: Це секретне повідомлення від Боба до Аліси

Боб шифрує повідомлення для Аліси:
[>] Підготовка текстового повідомлення: 42 символів
[i] Створено 2 блоків
[+] Шифрування текстового повідомлення з 2 блоками
[+] Шифрування завершено
Зашифровані блоки: [4246546248080924590960302295188001352165036127844190706689353514821897840292861421722158302757386185237527316895983963610486034565119884628971470575752438, 4642128812767387499601255443590611526284967153339126195561481758607855509223639442637791435928415796745992305864321646502748982437964131402129918408146869]

Аліса розшифровує повідомлення:
[-] Розшифрування 2 блоку (-ів)

Дані для перевірки в RSA калькуляторі:
-----

Блок 1:
С (зашифроване повідомлення) = 4246546248080924590960302295188001352165036127844190706689353514821897840292861421722158302757386185237527316895983963610486034565119884628971470575752438
N (модуль) = 8665378168296147583656121256803980008165915076604119432213102574382502074727541767306183136520669113134029351292040481463349232156107057313423381785584133
D (секретний ключ) = 364732070086285962273454849732117412339986217538374711289193027624611013215975713024934062520976607052530514183782968695197910499282708294603050475315213
Е (відкритий ключ) = 65537
Розшифроване значення = 42687391261479592576231314137534221934109176993905962913414946383426805363471682533612141100695661613876731609812700203870167794287142383660087282544672
-----
```

```
Блок 2:
С (зашифроване повідомлення) = 46421288127673874996012554435906115262849671533391261955614817586078555092236394426377914359284157967459923058643216465027489824379
64131402129918408146869
N (модуль) = 86653781682961475836561212568039800081659150766041194322131025743825020747275417673061831365206691131340293512920404814633492321561070573134233817855
84133
D (секретний ключ) = 364732070086285962273454849732117412339986217538374711289193027624611013215975713024934062520976607075253051418378296869519791049928270829460
3050475315213
Е (відкритий ключ) = 65537
Розшифроване значення = 426985797180327223963692137285881000242956684504420346977891714473965101549484360288744319086194794918541878560789499687758360483484941309
62403902357504
-----
[=] Успішно декодований текст: 42 символів
Розшифроване повідомлення: Це секретне повідомлення від Боба до Аліси
Перевірка успішна: True
```

Тест №4.2 - Аліса Бобу

```
Тест 4.2 - Аліса відповідає Бобу:
Оригінальне повідомлення: Re: Це секретне повідомл... Отримано!

Аліса шифрує повідомлення для Боба:
[>] Підготовка текстового повідомлення: 37 символів
[i] Створено 1 блоків
[+] Шифрування текстового повідомлення з 1 блоками
[+] Шифрування завершено
Зашифровані блоки: [469850905235379834501734227559022734364500166838719452579122249664307359848000597726690359211946494763796207043548553393874411274237412769942
996183123949]

Боб розшифровує повідомлення:
[-] Розшифрування 1 блоку (-1в)

Дані для перевірки в RSA калькуляторі:
-----

Блок 1:
С (зашифроване повідомлення) = 46985090523537983450173422755902273436450016683871945257912224966430735984800059772669035921194649476379620704354855339387441127423
74127699422996183123049
N (модуль) = 70851415037269785550016601043577883445129424893642690999419753745370181336761331624387978669916068687377995499928352378606282565773591011698381458321
29181
D (секретний ключ) = 542707330953651103134234611347423554472358687407245233710861290266198193860770913722267456921022003179966587046075409869543554262359897829105
49360202217
Е (відкритий ключ) = 65537
Розшифроване значення = 168570242575581172793155121835754831364081429935081092237843554391814847231526935803937456553782691515528679355768549513101784469892326689
00843921620513
-----
[=] Успішно декодований текст: 37 символів
Розшифроване повідомлення: Re: Це секретне повідомл... Отримано!
Перевірка успішна: True

=====
```

Тест №5.1 - Боб Алісі

```
Тест 5.1 - Боб надсилає Алісі:
Опис: Тест спеціальних символів
Оригінальне повідомлення: Test message with special chars: @#$%^&*()

Боб шифрує повідомлення для Аліси:
[>] Підготовка текстового повідомлення: 42 символів
[i] Створено 1 блоків
[+] Шифрування текстового повідомлення з 1 блоками
[+] Шифрування завершено
Зашифровані блоки: [6079962678574559603786487241945254314569320761559040256859953113595914149495381229718028051556116169118410826114994254439800024219205437143138
030088460588]

Аліса розшифровує повідомлення:
[-] Розшифрування 1 блоку (-1в)

Дані для перевірки в RSA калькуляторі:
-----

Блок 1:
С (зашифроване повідомлення) = 60799626785745596037864872419452543145693207615590402568599531135959141494953812297180280515561161691184108261149942544398000242192
05437143138030088460588
N (модуль) = 86653781682961475836561212568039800081659150766041194322131025743825020747275417673061831365206691131340293512920404814633492321561070573134233817855
84133
D (секретний ключ) = 364732070086285962273454849732117412339986217538374711289193027624611013215975713024934062520976607075253051418378296869519791049928270829460
3050475315213
Е (відкритий ключ) = 65537
Розшифроване значення = 172663770391419600443415473315273869193069734958445184643906771833043887486222162553898788701137319359505343946120164043814165490363895654
65854289641472
-----
[=] Успішно декодований текст: 42 символів
Розшифроване повідомлення: Test message with special chars: @#$%^&*()
Перевірка успішна: True
```

Тест №5.2 - Аліса Бобу

```
Тест 5.2 - Аліса відповідає Бобу:
Оригінальне повідомлення: Re: Test message with sp... Отримано!

Аліса шифрує повідомлення для Боба:
[>] Підготовка текстового повідомлення: 37 символів
[!] Створено 1 блоків
[+] Шифрування текстового повідомлення з 1 блоками
[+] Шифрування завершено
Зашифровані блоки: [1580920099536799948568116190661456040649520818207631204880115838284303801478268564324595229110468153606711653572056821210816720575096442678940500948581217]

Боб розшифровує повідомлення:
[-] Розшифрування 1 блоку (-ів)

Дані для перевірки в RSA калькуляторі:
-----
Блок 1:
С (зашифроване повідомлення) = 1580920099536799948568116190661456040649520818207631204880115838284303801478268564324595229110468153606711653572056821210816720575096442678940500948581217
N (модуль) = 708514150372697855500166010435778834451294248936426909994197537453701813362438797866991606868737799549992835237860628256577359101169838145832129181
D (секретний ключ) = 54270733095365110313423461134742355447235868740724523371086129026619819386077091372226745692102200317996658704607540986954355426235989782910549360202217
E (відкритий ключ) = 65537
Розшифроване значення = 16857024251639326175594467102097459512824575894141259120982275178775723300400330521724649176798382663166697495813466666458036251218325224587503226322944
-----
[=] Успішно декодований текст: 37 символів
Розшифроване повідомлення: Re: Test message with sp... Отримано!
Перевірка успішна: True
=====
```

Пояснення виводу тесту №2:

Вивід показує, що:

- **Числові повідомлення (Тести 1-2):**
 - ✧ Малі та великі числа успішно шифруються
 - ✧ Всі перевірки проходять успішно
 - ✧ Точне відновлення оригінальних значень
- **Текстові повідомлення (Тести 3-5):**
 - ✧ Коректна обробка кирилиці та емодзі
 - ✧ Правильне розбиття на блоки для довгих повідомлень
 - ✧ Спецсимволи обробляються без помилок
- **Шифрування/Розшифрування:**
 - ✧ Всі блоки коректно шифруються та розшифровуються
 - ✧ Формат даних зберігається
 - ✧ Перевірки успішні для всіх тестів
 - ✧ Коректний двосторонній обмін

Всі тести пройдені успішно, система працює коректно.

Блок/тест №3 | Цифровий підпис

```
95 def demonstrate_signature(alice_keys: Tuple[RSAPublicKey, RSAPrivateKey], bob_keys: Tuple[RSAPublicKey, RSAPrivateKey]):
96     print_section("ДЕМОНСТРАЦІЯ ЦИФРОВИХ ПІДПИСІВ")
97
98     alice_public, alice_private = alice_keys
99     bob_public, bob_private = bob_keys
100
101     test_messages = [
102         (12345, "Просте числове повідомлення"),
103         (987654321, "Велике числове повідомлення"),
104         ("Цей документ засвідчує справжність електронного підпису", "Текстове повідомлення")
105     ]
106
107     for i, (message, description) in enumerate(test_messages, 1):
108         print(f"\nТест {i}.1 - Підпис від Аліси:")
109         print(f"Опис: {description}")
110         print(f"Оригінальне повідомлення: {message}")
111
112         # створення підпису Алісою
113         print("\nАліса створює цифровий підпис:")
114         alice_signature = sign(message, alice_private)
115         print(f"Підпис: {alice_signature}")
116
117         # перевірка підпису Бобом
118         print("\nБоб перевіряє підпис Аліси:")
119         is_valid = verify(message, alice_signature, alice_public)
120         print(f"Підпис валідний: {is_valid}")
121
122     def demonstrate_signature(alice_keys: Tuple[RSAPublicKey, RSAPrivateKey], bob_keys: Tuple[RSAPublicKey, RSAPrivateKey]):
123         # перевірка на підробку
124         print("\nПеревірка виявлення підробки:")
125         if isinstance(message, str):
126             fake_message = message + " ПІДРОБЛЕНЮ"
127         else:
128             fake_message = message + 1
129         print(f"Змінене повідомлення: {fake_message}")
130
131         is_valid = verify(fake_message, alice_signature, alice_public)
132         print(f"Підпис для зміненого повідомлення валідний: {is_valid}")
133
134         # боб відповідає
135         reply_message = message * 2 if isinstance(message, int) else f"Отримав повідомлення: {message}..."
136         print(f"\nТест {i}.2 - Боб відповідає підписаним повідомленням:")
137         print(f"Повідомлення відповіді: {reply_message}")
138
139         # створення підпису Бобом
140         print("\nБоб створює цифровий підпис:")
141         bob_signature = sign(reply_message, bob_private)
142         print(f"Підпис Боба: {bob_signature}")
143
144         # Аліса перевіряє підпис Боба
145         print("\nАліса перевіряє підпис Боба:")
146         is_valid = verify(reply_message, bob_signature, bob_public)
147         print(f"Підпис валідний: {is_valid}")
148
149     print("\n" + "-"*50)
```

Ф-ія demonstrate_signature тестує створення і перевірку цифрових підписів через:

- **Підпис Аліси** (Створення підпису для повідомлення => Перевірка підпису Бобом => Тест на підробку повідомлення).
- **Підпис Боба** (Формування відповіді => Створення підпису => Перевірка Алісою).

Тестуються різні типи повідомлень: числа та текст.

Вивід тесту №3 | Цифровий підпис (інший ключ, бо перезапускав код):

Тест №1.1 - Аліса Бобу

```
=====
ДЕМОНСТРАЦІЯ ЦИФРОВИХ ПІДПИСІВ
=====

Тест 1.1 - Підпис від Аліси:
Опис: Просте числове повідомлення
Оригінальне повідомлення: 12345

Аліса створює цифровий підпис:
[i] Підписання повідомлення з 1 блоками
[i] Блок 1 підписано: 5843770043456519975937107049977101524571470395164922720771674597638653149078671707327818008961910095011255394042593399354012683853112019912787134512681933
Підпис: [5843770043456519975937107049977101524571470395164922720771674597638653149078671707327818008961910095011255394042593399354012683853112019912787134512681933]

Боб перевіряє підпис Аліси:
[v] Перевірка 1 підписів
[i] Підпис блоку 1 перевірено
[=] Всі підписи успішно перевірено
Підпис валідний: True

Перевірка виявлення підробки:
Змінене повідомлення: 12346
[v] Перевірка 1 підписів
[v] Перевірка підпису для блоку 1 не вдалася
Підпис для зміненого повідомлення валідний: False
```

Тест №1.2 - Боб Алісі

```
Тест 1.2 - Боб відповідає підписаним повідомленням:
Повідомлення відповіді: 24690

Боб створює цифровий підпис:
[i] Підписання повідомлення з 1 блоками
[i] Блок 1 підписано: 3873989922051593697467980207945157926824043604823258686193695366951850103683794886438564629425313073147081702275026725904760852602987750019301604606223606
Підпис Боба: [3873989922051593697467980207945157926824043604823258686193695366951850103683794886438564629425313073147081702275026725904760852602987750019301604606223606]

Аліса перевіряє підпис Боба:
[v] Перевірка 1 підписів
[i] Підпис блоку 1 перевірено
[=] Всі підписи успішно перевірено
Підпис валідний: True

=====
```

Тест №2.1 - Аліса Бобу

```
Тест 2.1 - Підпис від Аліси:
Опис: Велике числове повідомлення
Оригінальне повідомлення: 987654321

Аліса створює цифровий підпис:
[i] Підписання повідомлення з 1 блоками
[i] Блок 1 підписано: 7457381230254083537594256526009652513421436763780555631898156837391456706048936938656907256320060377733101379478257890839163467051408840324300307236669430
Підпис: [7457381230254083537594256526009652513421436763780555631898156837391456706048936938656907256320060377733101379478257890839163467051408840324300307236669430]

Боб перевіряє підпис Аліси:
[v] Перевірка 1 підписів
[i] Підпис блоку 1 перевірено
[=] Всі підписи успішно перевірено
Підпис валідний: True

Перевірка виявлення підробки:
Змінене повідомлення: 987654322
[v] Перевірка 1 підписів
[v] Перевірка підпису для блоку 1 не вдалася
Підпис для зміненого повідомлення валідний: False
```

Тест №2.2 - Боб Алісі

```
Тест 2.2 - Боб відповідає підписаним повідомленням:
Повідомлення відповіді: 1975308642

Боб створює цифровий підпис:
[i] Підписання повідомлення з 1 блоками
[i] Блок 1 підписано: 416907301195721954911923636410902856409491381279970582997200185683284552094509941363498469287384077727255405698908892969100243003684081879292192646874551
Підпис Боба: [416907301195721954911923636410902856409491381279970582997200185683284552094509941363498469287384077727255405698908892969100243003684081879292192646874551]

Аліса перевіряє підпис Боба:
[v] Перевірка 1 підписів
[i] Підпис блоку 1 перевірено
[=] Всі підписи успішно перевірено
Підпис валідний: True

=====
```

Тест №3.1 - Аліса Бобу

```
Тест 3.1 - Підпис від Аліси:
Опис: Текстове повідомлення
Оригінальне повідомлення: Цей документ засвідчує справжність електронного підпису

Аліса створює цифровий підпис:
[>] Підготовка текстового повідомлення: 55 символів
[i] Створено 2 блоків
[i] Підписання повідомлення з 2 блоками
[i] Блок 1 підписано: 58348842314779470497299488118145410313595842294131997369707862306447551922307003196982169885728588251616990057953465147629443027579028136600
02759825228199
[i] Блок 2 підписано: 83381073290112108711359137375447655111926955789435534284601457139870715434188256999898134726724461001141828933483477939683444419504455344705
42374404690358
Підпис: [583488423147794704972994881181454103135958422941319973697078623064475519223070031969821698857285882516169900579534651476294430275790281366000275982522819
9, 8338107329011210871135913737544765511192695578943553428460145713987071543418825699989813472672446100114182893348347793968344441950445534470542374404690358]

Боб перевіряє підпис Аліси:
[>] Підготовка текстового повідомлення: 55 символів
[i] Створено 2 блоків
[v] Перевірка 2 підписів
[i] Підпис блоку 1 перевірено
[i] Підпис блоку 2 перевірено
[=] Всі підписи успішно перевірено
Підпис валідний: True

Перевірка виявлення підробки:
Змінене повідомлення: Цей документ засвідчує справжність електронного підпису ПІДРОБЛЕНО
[>] Підготовка текстового повідомлення: 66 символів
[i] Створено 2 блоків
[v] Перевірка 2 підписів
[i] Підпис блоку 1 перевірено
[v] Перевірка підпису для блоку 2 не вдалася
Підпис для зміненого повідомлення валідний: False
```

Тест №3.2 - Боб Алісі

```
Тест 3.2 - Боб відповідає підписаним повідомленням:
Повідомлення відповіді: Отримав повідомлення: Цей документ засвідчує справжність електронного підпису...

Боб створює цифровий підпис:
[>] Підготовка текстового повідомлення: 80 символів
[i] Створено 3 блоків
[i] Підписання повідомлення з 3 блоками
[i] Блок 1 підписано: 23211347496897632046907153242057795453533882001124558153676075351355207052621845716659720571994413585548433702626044023221188204335504161256
6128459637583
[i] Блок 2 підписано: 42767683004000473509111132792548131539827470619166876094213902998106090721520612797029515792588786063456932590760721609995577768219343105669
89904251534688
[i] Блок 3 підписано: 68756125689930427762770796699944908500867955292687645418876782943257491729234540089893709640039015677620268594343432752058996845444520584607
74097024418160
Підпис Боба: [2321134749689763204690715324205779545353388200112455815367607535135520705262184571665972057199441358554843370262604402322118820433550416125661284596
37583, 4276768300400047350911113279254813153982747061916687609421390299810609072152061279702951579258878606345693259076072160999557776821934310566985904251534688,
6875612568993042776277079669994490850086795529268764541887678294325749172923454008989370964003901567762026859434343275205899684544452058460774097024418160]

Аліса перевіряє підпис Боба:
[>] Підготовка текстового повідомлення: 80 символів
[i] Створено 3 блоків
[v] Перевірка 3 підписів
[i] Підпис блоку 1 перевірено
[i] Підпис блоку 2 перевірено
[i] Підпис блоку 3 перевірено
[=] Всі підписи успішно перевірено
Підпис валідний: True

=====
```

Пояснення виводу тесту №3:

Вивід показує, що:

- Для числових повідомлень (Тести 1-2):
 - ✧ Успішне підписання та перевірка
 - ✧ Виявлення модифікацій (навіть +1)
 - ✧ Коректна обробка великих чисел
- Для текстового повідомлення (Тест 3):
 - ✧ Правильне розбиття на блоки
 - ✧ Створення окремих підписів для кожного блоку
 - ✧ Виявлення підробки при зміні довжини повідомлення

Всі тести пройшли успішно, система підписів працює коректно та виявляє будь-які модифікації повідомлень.

Блок/тест №4 | Протокол обміну ключами

```
150 def demonstrate_key_exchange(alice_keys: Tuple[RSAPublicKey, RSAPrivateKey], bob_keys: Tuple[RSAPublicKey, RSAPrivateKey]):
151     print_section("ПРОТОКОЛ ОБМІНУ КЛЮЧАМИ")
152
153     alice_public, alice_private = alice_keys
154     bob_public, bob_private = bob_keys
155
156     # попередження
157     if alice_private.n >= bob_public.n:
158         print("\nПопередження: Для оптимальної безпеки модуль Аліси повинен бути менший за модуль Боба")
159         print("Продовжуємо демонстрацію...\n")
160
161     # генерація секретного ключа
162     secret_key = 12345
163     if secret_key >= alice_private.n:
164         raise ValueError(f"Ключ k={secret_key} занадто великий для даної криптосистеми")
165
166     print(f"1. Початкові дані:")
167     print(f"    Секретний ключ k = {secret_key}")
168     print(f"    Відкритий ключ Аліси (e, n) = ({alice_public.e}, {alice_public.n})")
169     print(f"    Відкритий ключ Боба (e_1, n_1) = ({bob_public.e}, {bob_public.n})")
170
171     print("\n2. Аліса формує повідомлення:")
172
173     # 1: шифрування ключа
174     k1 = mod_pow(secret_key, bob_public.e, bob_public.n)
175     print(f"\n    а) Шифрування ключа k відкритим ключем Боба:")
176     print(f"        k_1 = k^(e_1) mod n_1")
177     print(f"        k_1 = {secret_key}^{bob_public.e} mod {bob_public.n}")
178     print(f"        k_1 = {k1}")
179
180     # 2: створення підпису
181     signature = mod_pow(secret_key, alice_private.d, alice_private.n)
182     print(f"\n    б) Створення цифрового підпису своїм закритим ключем:")
183     print(f"        S = k^d mod n")
184     print(f"        S = {secret_key}^{alice_private.d} mod {alice_private.n}")
185     print(f"        S = {signature}")
186
187     # 3: шифрування підпису
188     s1 = mod_pow(signature, bob_public.e, bob_public.n)
189     print(f"\n    в) Шифрування підпису відкритим ключем Боба:")
190     print(f"        S_1 = S^(e_1) mod n_1")
191     print(f"        S_1 = {signature}^{bob_public.e} mod {bob_public.n}")
192     print(f"        S_1 = {s1}")
193
194     print("\n3. Передача даних:")
195     print(f"    Аліса надсилає Бобу пару чисел (k_1, S_1) = ({k1}, {s1})")
196
197     print("\n4. Боб виконує перевірку отриманих даних:")
198
199     # 4: розшифрування ключа
200     received_k = mod_pow(k1, bob_private.d, bob_private.n)
201     print(f"\n    а) Розшифрування k_1 своїм закритим ключем:")
202     print(f"        k = k_1^(d_1) mod n_1")
203     print(f"        k = {k1}^{bob_private.d} mod {bob_private.n}")
204     print(f"        k = {received_k}")
```

```

150 def demonstrate_key_exchange(alice_keys: Tuple[RSAPublicKey, RSAPrivateKey], bob_keys: Tuple[RSAPublicKey, RSAPrivateKey]):
206     # 5: розшифрування підпису
207     received_signature = mod_pow(s1, bob_private.d, bob_private.n)
208     print("\n    б) Розшифрування s_1 своїм закритим ключем:")
209     print(f"        S = s_1^(d_1) mod n_1")
210     print(f"        S = {s1}^{bob_private.d} mod {bob_private.n}")
211     print(f"        S = {received_signature}")
212
213     # 6: перевірка підпису
214     verified_k = mod_pow(received_signature, alice_public.e, alice_public.n)
215     print("\n    в) Перевірка підпису відкритим ключем Аліси:")
216     print(f"        k ?= S^e mod n")
217     print(f"        k ?= {received_signature}^{alice_public.e} mod {alice_public.n}")
218     print(f"        {verified_k} ?= {received_k}")
219
220     # фінальна перевірка
221     signature_valid = (verified_k == received_k == secret_key)
222     print("\n5. Результати перевірки:")
223     print(f"    Отриманий ключ співпадає з відправленим: {received_k == secret_key}")
224     print(f"    Підпис підтверджує справжність ключа: {verified_k == received_k}")
225     print(f"    Загальний результат: {'Протокол успішно завершено' if signature_valid else 'Помилка протоколу'}")

```

Ф-ія demonstrate_key_exchange показує протокол обміну ключами **RSA** (попередньо є перевірка розміру секретного ключа):

- **Кроки Аліси:** (Шифрування ключа k ключем Боба $\Rightarrow$ Створення підпису своїм приватним ключем $\Rightarrow$ Шифрування підпису ключем Боба).
- **Кроки Боба:** (Розшифрування k своїм ключем $\Rightarrow$ Розшифрування підпису $\Rightarrow$ Верифікація підпису ключем Аліси).

Вивід тесту №4 | Протокол обміну ключами:

вже для іншої пари ключів, бо випадково закрит консоль

Початкові дані:

```

=====
ПРОТОКОЛ ОБМІНУ КЛЮЧАМИ
=====

Попередження: Для оптимальної безпеки модуль Аліси повинен бути менший за модуль Боба
Продовжуємо демонстрацію...

1. Початкові дані:
   Секретний ключ k = 12345
   Відкритий ключ Аліси (e, n) = (65537, 866537816829614758365612125680398008165915076604119432213102574382502074727541767306183136520669113134029351292040481463
349232156107057313423381785584133)
   Відкритий ключ Боба (e_1, n_1) = (65537, 7085141503726978555001660104357788344512942489364269099941975374537018133676133162438797866991606868737799549992835237
860628256577359101169838145832129181)

```

Аліса формує повідомлення:

```

2. Аліса формує повідомлення:

а) Шифрування ключа k відкритим ключем Боба:
   k_1 = k^(e_1) mod n_1
   k_1 = 12345^65537 mod 70851415037269785550016601043577883445129424893642690999419753745370181336761331624387978669916068687377995499928352378606282565773591
01169838145832129181
   k_1 = 897593691664869334655938952787061690364366485967931812776961008344704543518442852772286115293449180101527551706024008527087835678616909145904289708
51

б) Створення цифрового підпису своїм закритим ключем:
   s = k^d mod n
   s = 12345^3647320708062859622734548497321174123398621753837471128919302762461101321597571302493406252097660707525305141837829686951979104992827082046030504
75315213 mod 8665378168296147583656121256803980081659150766041194322131025743825020747275417673061831365206691131340293512920404814633492321561070573134233817855
84133
   s = 58437700434565199759371070499771015245714703951649227207716745976386531490786717073278180089619100950112553940425933993540126838531120199127871345126819
33

в) Шифрування підпису відкритим ключем Боба:
   s_1 = S^(e_1) mod n_1
   s_1 = 584377004345651997593710704997710152457147039516492272077167459763865314907867170732781800896191009501125539404259339935401268385311201991278713451268
1933^65537 mod 708514150372697855500166010435778834451294248936426909994197537453701813367613316243879786699160686873779954999283523786062825657735910116983814583
2129181
   s_1 = 38659885518858654389531323959306347371673775546803031323947027034702007708325329330685614375401414103258644788172614646990398106465508475286744670094
6211

```

Передача даних:

```
3. Передача даних:
Аліса надсилає Бобу пару чисел (k_1, S_1) = (897593691664869334655938952787061690364364366485967931812776961008344704543518442852772286115293449180101527551706
02400852708783567861690814590428970051, 38659885518858654389531323959306347371673775554680303132339470270347020077083253293306856143754014141032586447881726144699
03981064655084752867446700946211)
```

Боб виконує перевірку отриманих даних:

```
4. Боб виконує перевірку отриманих даних:

а) Розшифрування k_1 своїм закритим ключем:
k = k_1^(d_1) mod n_1
k = 89759369166486933465593895278706169036436436648596793181277696100834470454351844285277228611529344918010152755170602400852708783567861690914590428970051
^54270733095365110313423461134742355447235868740724523371086129026619819386077091372226745692102200317996658704607540986954355426235989782910549360202217 mod 70
141503726978555001660104357788344512942489364269099941975374537018133676133162438797866991606868737799549992835237860628256577359101169838145832129181
k = 12345

б) Розшифрування S_1 своїм закритим ключем:
S = S_1^(d_1) mod n_1
S = 38659885518858654389531323959306347371673775554680303132339470270347020077083253293306856143754014141032586447881726144699039810646550847528674467009462
11^54270733095365110313423461134742355447235868740724523371086129026619819386077091372226745692102200317996658704607540986954355426235989782910549360202217 mod 70
85141503726978555001660104357788344512942489364269099941975374537018133676133162438797866991606868737799549992835237860628256577359101169838145832129181
S = 58437700434565199759371070499771015245714703951649227207716745976386531490786717073278180089619100950112553940425933993540126838531120199127871345126819
33

в) Перевірка підпису відкритим ключем Аліси:
k ?= S^e mod n
k ?= 5843770043456519975937107049977101524571470395164922720771674597638653149078671707327818008961910095011255394042593399354012683853112019912787134512681
933^65537 mod 8665378168296147583656121256803980008165915076604119432213102574382502074727541767306183136520669113134029351292040481463349232156107057313423381785
584133
12345 ?= 12345
```

Результати перевірки

```
5. Результати перевірки:
Отриманий ключ співпадає з відправленим: True
Підпис підтверджує справність ключа: True
Загальний результат: Протокол успішно завершено

ДЕМОНСТРАЦІЯ ЗАВЕРШЕНА
```

Пояснення виводу тесту№4:

Вивід показує, що:

- **Аліса правильно сформувала повідомлення** (Зашифрувала ключ k=12345 ключем Боба => Створила підпис => Зашифрувала підпис)
- **Боб успішно розшифрував і отримав оригінальний ключ (12345), розшифрував підпис та підтвердив автентичність через перевірку підпису.**
- **Всі перевірки пройшли успішно:**
 - ✧ Ключ відновлено правильно
 - ✧ Підпис підтверджено
 - ✧ Протокол завершено без помилок

Протокол виконав безпечну передачу ключа з підтвердженням автентичності відправника.

Висновки

В результаті виконання даної лабораторної роботи було розроблено повноцінну криптосистему RSA, яка включає генерацію ключів на основі безпечних простих чисел, механізми шифрування та розшифрування різних типів повідомлень, створення цифрових підписів та протокол безпечного обміну ключами. В коді використовується тест Міллера-Рабіна для перевірки простих чисел та стандартна відкрита експонента ($e=65537$).

Тестування коду підтвердило його правильність та коректність при роботі з різними типами даних, включаючи числові та текстові повідомлення, кирилицю, емодзі та спеціальні символи.

Створений механізм розбиття на блоки дозволяє обробляти повідомлення довільної довжини, а система підписів коректно виявляє будь-які “підробки” даних.

Всі тести пройшли успішно, підтверджуючи коректність роботи розробленого коду.